Bits y hexadecimal

El byte son 8 bits. Se cuentan del 0 al 7 empezando por la derecha. Los pesos son 1, 2, 4, 8, 16, 32, 64, 128.

00100000 = 32, en hexadecimal 0x20.

10000001 = 129 = 0x81.

0xFF vale 255 y 0x08 señala el bit 3.

Hexadecimal: 0 al 9, después A, B, C, D, E, F. Cada dígito hex ocupa 4 bits. Sirve mucho para escribir registros sin poner tantos ceros.

Uno suele manejar 5 V como nivel alto, ESP32 3,3 V. Son niveles lógicos, no es que el chip solo conozca 0 V y el voltaje exacto.

Operaciones con bits

AND: tienen que ser 1 los dos. OR: basta que uno sea 1. XOR: uno diferente del otro. NOT invierte.

0xCA & 0x0F = 0x0A, se queda con los cuatro bits de abajo.

Para seleccionar una posición usamos máscara. 1 << 5 da 00100000, osea bit 5.

REG |= (1 << n) para prender un bit.

REG &= ~(1 << n) para apagarlo.

REG ^= (1 << n) lo cambia.

REG & (1 << n) para ver si está activo.

Ojo con REG = (1 << 2), ahí reemplazo todo el valor y se me borran los otros bits. Con |= conservo los que ya estaban.

Si el registro era 10101010 y debo cambiar solo una posición, no es necesario escribir los 8 bits otra vez.

Registros

Una dirección especial de memoria puede servir para controlar un periférico. Escribir ahí realmente puede hacer que un pin cambie.

En AVR el programa C termina convertido en instrucciones máquina. LDI carga, ADD suma, AND, OR, EOR; SBI pone un bit y CBI lo borra. RJMP hace salto.

RISC instrucciones simples, CISC más complejas. El AVR es de 8 bits, entonces cuando hay números mayores debe hacer más operaciones.

PORTB |= (1 << 5) en el ejemplo puede convertirse en una instrucción SBI. No es lo mismo escribir el registro entero.

DDR, PORT, PIN

DDR decide la dirección del pin. 1 significa salida, 0 entrada.

Cuando es salida, PORT con 1 lo pone HIGH, con 0 LOW. Si es entrada, PORT con 1 activa la resistencia pull-up interna del AVR. PIN sirve para leer el valor que tiene el pin.

DDRB5 = 1, el PB5 trabaja como salida. PORTB5 = 1 entonces sale alto.

Si DDR=0 y PORT=0 queda como entrada sin pull-up. Si nadie conecta nada, el valor puede quedar flotando.

DDR=0 y PORT=1 es entrada con pull-up. El botón a GND, al soltar se lee 1; cuando se presiona se lee 0. La lógica es invertida.

Entrada flotante: no es un cero fijo ni un uno fijo. Puede cambiar por ruido.

Para un LED hace falta resistencia, no conectarlo así nomás.

Con Arduino: pinMode(13, OUTPUT) en Uno configura DDRB5. digitalWrite(13, HIGH) modifica PORTB5. INPUT_PULLUP deja entrada y habilita resistencia.

La diferencia es que Arduino lo hace con funciones y el registro se puede manejar directamente.

En un ejercicio: 0x81 = 10000001. Para quitar bit 4, PORTB &= ~(1 << 4). Si el botón usa pull-up y está presionado, lo que se lee

También preguntar por qué un cambio con |= conserva otros bits y con = no.